

Topic 1: Loading Dataset

- Loading a dataset is the first and most important step before performing any kind of data analysis. The dataset consists of personal email data that is downloaded from Gmail in MBOX file format.
- The MBOX file stores all email messages together in a single file, and it is commonly used for email analysis.
- Before loading the dataset, we must ensure that the required Python libraries are available.

Importing required libraries

The following Python libraries:

- mailbox – to read the MBOX email file
- numpy – for numerical operations
- pandas – for handling structured data
- matplotlib – for visualization (used later)

Code

```
import mailbox
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

Loading the MBOX email file

- Once the libraries are imported, the next step is to load the downloaded MBOX file into Python.
- The MBOX file path must be provided correctly, otherwise the dataset cannot be read.

Code

```
mboxfile = "PATH TO DOWNLOADED MBOX FILE"
mbox = mailbox.mbox(mboxfile)
mbox
```

Understanding the loaded dataset

- After loading the dataset, it is important to understand what information is present inside each email.
- Each email message contains several keys (fields) such as sender, receiver, subject, date, and message ID.
- To view all available keys in the dataset, the textbook uses the following loop.

Code

```
for key in mbox[0].keys():  
    print(key)
```

Output (keys present in dataset)

The dataset contains keys such as:

- Message-ID
- Date
- From
- To
- Subject
- MIME-Version
- Content-Type
- X-Mailer
- Received
- Authentication-Results

Why loading the dataset is important

- It allows Python to access raw email data
- It helps in identifying what fields are available
- It is the foundation for data cleansing, transformation, and analysis
- Without loading the dataset properly, EDA (Exploratory Data Analysis) cannot be performed

Topic 2: Data Transformation

After loading the email dataset, the extracted data contains many fields that are not required for analysis. Data transformation is the process of selecting, converting, cleaning, and restructuring data so that it becomes suitable for analysis.

Data transformation is the process of converting raw extracted data into a clean, structured, and usable format suitable for data analysis.

Need for Data Transformation

- The extracted email data contains raw and unorganized information
- Not all fields are useful for analysis
- Data must be converted into a tabular format
- Date and time values must be converted into proper datetime format
- Missing and unnecessary values must be removed

Creating a structured dataset

First convert raw email data into a **CSV file** that contains only the required attributes.

Code

```
import csv

with open('mailbox.csv', 'w') as outputfile:

    writer = csv.writer(outputfile)

    writer.writerow(['subject', 'from', 'date', 'to', 'label', 'thread'])

    for message in mbox:

        writer.writerow([

            message['subject'],

            message['from'],

            message['date'],

            message['to'],

            message['X-Gmail-Labels'],

            message['X-GM-THRID']

        ])
```

Loading transformed data

After creating the CSV file, it is loaded using **pandas** for further processing.

Code

```
df = pd.read_csv(  
    'mailbox.csv',  
    names=['subject', 'from', 'date', 'to', 'label', 'thread']  
)
```

Converting date field

Initially, the date column is stored as an **object type**. For time-based analysis, it must be converted into **datetime format**.

Checking data types

```
df.dtypes
```

Converting date to datetime

```
df['date'] = df['date'].apply(  
    lambda x: pd.to_datetime(x, errors='coerce', utc=True)  
)
```

Removing missing values

Some rows may contain **missing date values**, which can affect analysis.

Code

```
df = df[df['date'].notna()]
```

Saving transformed data

The cleaned and transformed data is saved into a new CSV file.

Code (from textbook)

```
df.to_csv('email.csv')
```

Applying descriptive statistics

After transformation, basic statistics are applied to verify data consistency.

Code

```
df.info()
```

After completing data transformation:

- Raw email data becomes clean and structured
- Unnecessary fields are removed
- Date and time values are converted properly
- Missing values are handled
- Dataset is ready for Exploratory Data Analysis (EDA)

Topic 3: Data Analysis

Data analysis is the most important stage of Exploratory Data Analysis (EDA). After loading, cleaning, and transforming the email dataset, data analysis is performed to extract useful insights from the data.

Data analysis in email data is used to answer questions such as:

- How many emails are sent during a given time period?
- At what times of the day emails are sent and received?
- What is the average number of emails per day and per hour?
- Which day of the week is the busiest?
- Which time of day is most active for email communication?

Number of emails in a given time period

To find how many emails are sent and received, the dataset is filtered based on **labels**:

- sent → outgoing emails
- inbox → incoming emails

Code

```
print(df.index.min().strftime('%a %d %b %Y %I:%M %p')) # for starting date
print(df.index.max().strftime('%a %d %b %Y %I:%M %p')) # for ending date
print(df['label'].value_counts())
```

Symbol	Meaning
%a	Abbreviated weekday name (Mon, Tue)

Symbol	Meaning
%d	Day of the month
%b	Abbreviated month name
%Y	Year
%I	Hour (12-hour format)
%M	Minutes
%p	AM or PM

Analyzing time of day for emails

It analyzes **when emails are sent and received during the day**.
Separate datasets are created for sent and received emails.

Code (as given in textbook)

```
sent = df[df['label'] == 'sent']
received = df[df['label'] == 'inbox']
```

Average emails per day and per hour

To understand email frequency, it calculates:

- Average number of emails per day
- Average number of emails per hour

Code

```
df['dayofweek'] = df['date'].apply(lambda x: x.weekday())
df['hour'] = df['date'].apply(lambda x: x.hour)
```

Number of emails per day

The dataset is grouped based on the day of the week to find the busiest day.

Code

```
counts = df['dayofweek'].value_counts(sort=False)
counts.plot(kind='bar')
```

Incoming vs outgoing emails per week

Then it compares sent and received emails to understand communication balance.

Code

```
dfw = sent.groupby('dayofweek').size() / len(sent)
dfw2 = received.groupby('dayofweek').size() / len(received)
```

Most active time of day

The textbook analyzes which hours of the day are most active for email communication.

Code

```
df_r = received.copy()
df_s = sent.copy()

df_r['hour'] = df_r['date'].dt.hour
df_s['hour'] = df_s['date'].dt.hour
```

Topic 4: Merging database-style DataFrames

Data transformation is an important step in data analysis where the structure of data is changed to make it suitable for further processing and analysis. One common transformation operation is merging database-style DataFrames.

In practical applications, data is rarely stored in a single table. Instead, related data is distributed across multiple tables. These tables are combined using keys in database systems. Pandas follows the same concept and allows combining multiple DataFrames using merge operations, which work similarly to SQL JOINS.

Database-style merging refers to the process of combining two or more DataFrames based on a common column or index. This common column is called a key.

- A key uniquely identifies rows in a DataFrame
- The same key must be present in both DataFrames
- Merging produces a new DataFrame
- Original DataFrames remain unchanged

Creation of DataFrames

To demonstrate merging, it uses two simple DataFrames that share a common key.

Code:

```
import pandas as pd

df1 = pd.DataFrame({
    'key': ['A', 'B', 'C'],
    'data1': [1, 2, 3]
})
```

```
df2 = pd.DataFrame({
    'key': ['A', 'B', 'D'],
    'data2': [4, 5, 6]
})
```

Merging DataFrames using a common column

The primary function used for merging is `pd.merge()`.

Code:

```
pd.merge(df1, df2, on='key')
```

Inner join (default behavior)

Inner join returns only the rows where the key exists in both DataFrames.

Example result:

key	data1	data2
A	1	4
B	2	5

Left join

```
pd.merge(df1, df2, on='key', how='left')
```

- Ensures no data from `df1` is lost

Right join

```
pd.merge(df1, df2, on='key', how='right')
```

- Ensures no data from df2 is lost

Outer join

```
pd.merge(df1, df2, on='key', how='outer')
```

Meaning:

- Provides a complete view of data

Merging using index values

The merging is done based on using index values instead of columns.

```
pd.merge(df1, df2, left_index=True, right_index=True)
```

- Uses index positions as merge keys
- No column name is required
- Useful for time-series and transformed datasets

Important characteristics of merging

- Merge operation does not modify original DataFrames
- Produces a new DataFrame
- Works like relational database joins
- Requires at least one common key or index
- Handles missing data using NaN

Merging database-style DataFrames is important because:

- It combines related datasets into one
- Enables feature enrichment
- Avoids duplication of data
- Supports complex analysis

- Makes exploratory data analysis more effective

Topic 5: Transformation techniques

Data transformation is the process of modifying the structure, format, and representation of data so that it becomes suitable for analysis. After loading and cleaning the dataset, transformation techniques are applied to convert raw data into meaningful and structured data.

Transformation techniques are used to:

- Create new variables from existing data
- Separate data based on conditions
- Extract useful information from date and time
- Combine related datasets
- Prepare data for exploratory data analysis

Creating new variables from date information

The email dataset contains a date column that stores both date and time. To perform time-based analysis, new variables are created from this column.

```
df['dayofweek'] = df['date'].apply(lambda x: x.weekday())
```

```
df['hour'] = df['date'].apply(lambda x: x.hour)
```

- `apply()` applies a function to each value in the column
- `weekday()` extracts the day of the week
0 → Monday, 6 → Sunday
- `hour` extracts the hour of the day (0–23)
- New columns `dayofweek` and `hour` are created
- These columns are used for time-based analysis

Separating data using conditions

Email data contains both sent and received emails. To analyze them separately, the dataset is divided using conditional filtering.

```
sent = df[df['label'] == 'sent']
```

```
received = df[df['label'] == 'inbox']
```

- The label column indicates the type of email
- Conditional filtering creates two separate DataFrames
- Helps analyze outgoing and incoming emails independently

Creating copies before further transformation

To avoid modifying original datasets, copies are created before applying additional transformations.

```
df_r = received.copy()
```

```
df_s = sent.copy()
```

- `.copy()` creates independent copies
- Prevents accidental changes to original data
- Ensures safe transformation and analysis

Extracting hour using pandas datetime accessor

Instead of using `apply()`, the pandas datetime accessor can be used for efficient extraction.

```
df_r['hour'] = df_r['date'].dt.hour
```

```
df_s['hour'] = df_s['date'].dt.hour
```

- `.dt` accesses datetime properties
- Extracts hour directly from the date column
- Faster and cleaner than using `apply()`
- Used for hour-wise email analysis

Grouping transformed data

After creating new variables, the transformed data is grouped to summarize patterns.

```
dfw = sent.groupby('dayofweek').size() / len(sent)
```

```
dfw2 = received.groupby('dayofweek').size() / len(received)
```

- `groupby('dayofweek')` groups emails by weekday
- `size()` counts number of emails per day

- Division converts counts into proportions
- Used to compare weekly email activity

Merging transformed DataFrames

Transformation also includes combining related datasets using database-style merging.

```
pd.merge(df1, df2, on='key')
```

- Combines two DataFrames using a common key
- Similar to database join operation
- Produces a single transformed DataFrame

Handling Missing Values

During transformation and merging, some values may become missing. Missing values occur due to incomplete data or unmatched keys during merging. In pandas, missing values are represented as NaN.

Handling missing values is important to ensure accurate analysis.

Identifying missing values

```
df.isnull()
```

- `isnull()` returns True for missing values
- Helps detect which entries are missing

Counting missing values

```
df.isnull().sum()
```

- Counts the number of missing values in each column
- Helps understand the extent of missing data

Removing missing values

```
df.dropna()
```

- Removes rows containing NaN values

- Used when missing values are very few
- Produces a clean dataset

Filling missing values

`df.fillna(0)`

- Replaces missing values with 0
- Prevents data loss
- Useful when numeric values are missing

Grouping data for analysis

`df.groupby('dayofweek').size()`

- Groups data based on weekday
- Counts number of records per group
- Helps identify patterns

Normalizing grouped data

`df.groupby('dayofweek').size() / len(df)`

- Converts counts into proportions
- Allows fair comparison across days

After transformation and handling missing values:

- Data becomes clean and consistent
- New variables are meaningful
- Dataset is suitable for grouping and comparison
- Data is ready for exploratory data analysis

These steps are important because they:

- Improve data quality
- Convert raw data into structured form
- Prevent errors during analysis
- Enable meaningful interpretation

- Support exploratory data analysis

Topic 6: Renaming Axis Indexes

- Renaming axis indexes is a data transformation technique used to change the labels of rows and columns in a DataFrame.
- Axis indexes help in identifying data clearly and renaming them improves readability and understanding during analysis.
- In pandas, rows are called index (axis = 0) and columns are called columns (axis = 1).

Renaming axis indexes is useful to:

- Improve clarity of data
- Replace numeric or unclear labels
- Make output understandable for analysis
- Present results in a readable format
- Prepare data for reporting and visualization

Renaming row index values

Row indexes can be renamed using the `map()` function.

```
df.index = df.index.map(str.upper)
```

- `df.index` refers to row labels
- `map()` applies a function to each index value
- `str.upper` converts index labels to uppercase
- The row index values are transformed

Renaming column names

Column names can be renamed using the `rename()` function.

```
df.rename(columns={'old_name': 'new_name'})
```

- `columns` specifies column labels

- 'old_name' is the existing column
- 'new_name' is the updated column name
- Improves clarity of column meaning

Renaming both row and column indexes

```
df.rename(index={0: 'zero', 1: 'one'}, columns={'A': 'alpha'})
```

- index renames row labels
- columns renames column labels
- Both transformations happen together

Renaming axis using axis parameter

Axis labels can also be renamed using the axis parameter.

```
df.rename({0: 'zero', 1: 'one'}, axis='index')
```

```
df.rename({'A': 'alpha', 'B': 'beta'}, axis='columns')
```

- axis='index' → renames row labels
- axis='columns' → renames column labels
- Provides flexible control over axis transformation

In-place renaming of axis indexes

The renaming directly without creating a new DataFrame.

```
df.rename(columns={'A': 'alpha'}, inplace=True)
```

- inplace=True modifies the original DataFrame
- No new DataFrame is created
- Useful when permanent change is required

Renaming axis indexes:

- Makes data more readable
- Improves understanding of analysis output
- Helps in clear interpretation of results
- Supports effective exploratory data analysis

Topic 7: Discretization and Binning

- Discretization and binning are important data transformation techniques used to convert continuous numerical data into discrete categories or intervals.
- These techniques simplify data by reducing the number of distinct values and help in better understanding, analysis, and visualization.

Discretization is the process of transforming continuous-valued attributes into discrete-valued attributes.

- Continuous data can take any value within a range
- Discretization groups these values into finite intervals
- Each interval represents a category
- This helps convert numerical data into a form that is easier to analyze.

Binning is a specific discretization technique where values are grouped into bins (ranges or intervals).

- Each bin covers a specific range of values
- All values in a bin are treated as equivalent
- Reduces the complexity of data
- Binning is commonly used in exploratory data analysis.

Discretization and binning are used to:

- Reduce noise in data
- Simplify continuous data
- Improve readability of results
- Identify patterns more easily

- Support grouping and aggregation
- Make analysis results interpretable

Discretization and binning are important because they:

- Convert continuous data into categorical form
- Reduce the effect of small variations
- Make large datasets easier to understand
- Improve pattern detection
- Support grouping and aggregation operations
- Enhance exploratory data analysis

Equal-width binning

In equal-width binning, the total range of data is divided into equal-sized intervals.

```
pd.cut(df['hour'], bins=4)
```

- `pd.cut()` is a pandas function used for binning
- `df['hour']` is the continuous numerical data
- `bins=4` divides the range into four equal parts
- Each value is placed into one of the bins

Creating labeled bins

To make bins more meaningful, labels can be assigned.

```
pd.cut(df['hour'], bins=4, labels=['Night', 'Morning', 'Afternoon', 'Evening'])
```

- Each bin is given a descriptive name
- Improves clarity and understanding
- Makes the output suitable for reporting and visualization

Storing binned data in a new column

Binned values are usually stored in a new column so that original data is preserved.

```
df['hour_group'] = pd.cut(df['hour'], bins=4)
```

- A new column hour_group is created
- Original hour values remain unchanged
- The new column can be used for grouping and analysis

Frequency-based binning (quantile binning)

Frequency-based binning, where each bin contains approximately the same number of observations.

```
pd.qcut(df['hour'], 4)
```

- pd.qcut() divides data based on quantiles
- Each bin has nearly equal number of values
- Useful when data is unevenly distributed

Difference between cut() and qcut()

- cut() → divides data based on value range
- qcut() → divides data based on number of observations

Both methods help in discretizing data but are used in different situations.

Topic 8: Outlier detection and filtering

- Outlier detection and filtering is a data transformation technique used to identify and handle abnormal or extreme values in a dataset.
- An outlier is a data value that differs significantly from other observations and may affect the correctness of analysis.

An outlier is a data point that:

- Lies far away from most of the data values
- Does not follow the general data pattern

- May be caused by errors, noise, or unusual behavior
- Outliers can distort averages, trends, and analysis results.

Outlier detection is important to:

- Improve data quality
- Avoid misleading analysis results
- Reduce noise in data
- Make patterns clearer
- Ensure reliable exploratory data analysis

Identifying outliers using summary statistics

The textbook explains that **statistical measures** such as minimum, maximum, and quartiles help in detecting outliers.

`df.describe()`

- `describe()` provides summary statistics
- Shows minimum, maximum, mean, and quartiles
- Extremely high or low values indicate possible outliers

Outlier detection using Interquartile Range (IQR)

The Interquartile Range (IQR) method is a common technique used to detect outliers.

- $Q1 \rightarrow$ First quartile (25%)
- $Q3 \rightarrow$ Third quartile (75%)
- $IQR = Q3 - Q1$

Values outside the range are treated as outliers.

`Q1 = df['value'].quantile(0.25)`

`Q3 = df['value'].quantile(0.75)`

`IQR = Q3 - Q1`

- `quantile(0.25)` gives Q1
- `quantile(0.75)` gives Q3

- IQR represents the spread of middle data

Detecting outliers using IQR limits

```
outliers = df[(df['value'] < (Q1 - 1.5 * IQR)) |
              (df['value'] > (Q3 + 1.5 * IQR))]
```

- Values below $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$
- These values are considered outliers
- The result contains only abnormal values

Filtering (removing) outliers

Once detected, outliers can be filtered (removed) from the dataset.

```
df_filtered = df[(df['value'] >= (Q1 - 1.5 * IQR)) &
                  (df['value'] <= (Q3 + 1.5 * IQR))]
```

Explanation

- Keeps only valid values within limits
- Removes extreme values
- Produces a cleaner dataset

Filtering outliers:

- Improves accuracy of analysis
- Prevents skewed results
- Makes data more consistent
- Enhances visualization and interpretation

Topic 9: Permutation and Random Sampling

- Permutation and random sampling are data transformation techniques used to rearrange data randomly or select random subsets of data.
- These techniques are useful in exploratory data analysis (EDA) to understand data behaviour, test robustness, and avoid bias caused by data ordering.

Permutation refers to the process of reordering or shuffling the elements of a dataset randomly.

- All data values remain the same
- Only the order of data changes
- Used to remove any bias caused by original ordering
- Permutation helps in analyzing whether patterns depend on data order.

Permutation and random sampling are important because they:

- Reduce bias in data analysis
- Help validate patterns and trends
- Improve reliability of results
- Enable analysis on smaller data subsets
- Support exploratory data analysis

Permuting data values

The permutation is done using random reordering of values.

```
np.random.permutation(df['value'])
```

- `np.random.permutation()` randomly rearranges values
- Original data values are preserved
- Produces a new randomized sequence
- Used for unbiased analysis

Random sampling is the process of selecting a subset of data randomly from a dataset.

- Each data point has an equal chance of selection
- Reduces dataset size while preserving distribution
- Useful for quick analysis and testing

Random sampling of rows

The random sampling is done by using pandas.

`df.sample(n=5)`

- `sample()` selects rows randomly
- `n=5` selects five random rows
- Each execution may return different rows

Sampling using a fraction

Instead of a fixed number, a fraction of data can also be sampled.

`df.sample(frac=0.3)`

- `frac=0.3` selects 30% of the dataset
- Size depends on total number of rows
- Useful for large datasets

Random sampling with reproducibility

To get the same random sample every time, a random seed is used.

`df.sample(n=5, random_state=42)`

- `random_state` ensures reproducible results
- Same rows are selected in every run
- Important for experiments and comparisons

Topic 10: Computing Indicators

- Computing indicators is a data transformation technique used to create new variables that indicate the presence or absence of a specific condition.
- An indicator is usually a binary variable that takes values such as True/False or 1/0.

An indicator is a variable that:

- Represents whether a condition is met

- Is derived from an existing column
- Helps in classification and analysis
- Simplifies decision-making
- Indicators are often called dummy variables.

Computing indicators is useful to:

- Convert conditions into analyzable form
- Simplify complex data
- Support grouping and filtering
- Enable conditional analysis
- Improve clarity of results

Computing indicators is important because it:

- Converts conditions into usable variables
- Enables categorical analysis
- Simplifies filtering and grouping
- Improves interpretability of data
- Supports exploratory data analysis

Creating an indicator using a condition

Computing indicators by applying a logical condition to a column.

```
df['is_sent'] = df['label'] == 'sent'
```

- Checks whether the email is sent
- Returns True if condition is satisfied
- Returns False otherwise
- A new indicator column is_sent is created

Converting Boolean indicators into integers

Indicators can also be converted into numeric form.

```
df['is_sent'] = (df['label'] == 'sent').astype(int)
```

- True is converted to 1
- False is converted to 0
- Useful for counting and aggregation

Computing indicators using numerical conditions

Indicators can also be created from numerical data.

```
df['late_night'] = df['hour'] > 22
```

- Checks if email was sent after 10 PM
- Creates an indicator for late-night emails
- Useful for time-based analysis

Using indicators in grouping operations

Computed indicators are often used **for grouping and summarization**.

```
df.groupby('is_sent').size()
```

- Groups data based on indicator values
- Counts sent vs non-sent emails
- Helps compare categories easily

Topic 11: Dummy Variables

- Dummy variables are a data transformation technique used to convert categorical (non-numeric) data into numerical form.

- They are mainly used when categorical values need to be represented in a way that can be easily analyzed, grouped, or computed.

A dummy variable is a variable that:

- Represents a category
- Takes values 0 or 1
- Indicates the presence or absence of a condition
- Is derived from a categorical column
- Dummy variables are also known as indicator variables.

Dummy variables are required because:

- Many analysis operations require numerical data
- Categorical values like text cannot be directly computed
- Binary representation simplifies analysis
- Helps in grouping and comparison
- Makes data suitable for further processing

Dummy variables are important because they:

- Convert categorical data into numerical form
- Enable mathematical operations
- Support grouping and filtering
- Improve data interpretability
- Prepare data for analysis and modelling

Creating dummy variables from a categorical column

Dummy variables using pandas can be done as:

```
pd.get_dummies(df['label'])
```

- `get_dummies()` converts categories into binary columns
- Each unique category becomes a separate column

- Values are marked as 1 if the category is present, otherwise 0

Example output

inbox	sent
1	0
0	1

Adding dummy variables to the DataFrame

Dummy variables can be added to the original DataFrame.

```
dummies = pd.get_dummies(df['label'])
```

```
df = df.join(dummies)
```

- Dummy variables are created separately
- `join()` adds them to the original DataFrame
- Original data remains intact

Creating dummy variables with prefix

To clearly identify dummy columns, a prefix can be used.

```
pd.get_dummies(df['label'], prefix='type')
```

- Adds prefix to column names
- Improves clarity and readability
- Useful when multiple categorical columns exist

Dummy variables for multiple categories

When a column has more than two categories:

- Each category is converted into a separate dummy column
- Each row will have exactly one 1 and remaining 0s

- This avoids confusion and supports clean analysis.

Topic 12: String Manipulation

- String manipulation is a data transformation technique used to process, modify, and extract information from text (string) data.
- In many real-world datasets, data is stored in string format, such as names, email addresses, labels, and text fields.

String manipulation refers to operations performed on string (text) data to:

- Modify text values
- Extract useful parts of text
- Convert text into a standard format
- Clean unwanted characters
- These operations help convert raw textual data into meaningful information.

String manipulation is required to:

- Clean inconsistent text data
- Extract meaningful features from strings
- Standardize text for comparison
- Prepare data for analysis
- Improve data quality and readability

String manipulation is important because it:

- Cleans and standardizes text data
- Extracts useful features
- Improves data consistency
- Enables better grouping and filtering
- Supports exploratory data analysis

Converting text to lowercase and uppercase

To maintain consistency, strings are often converted to a common case.

```
df['label'] = df['label'].str.lower()
```

```
df['label'] = df['label'].str.upper()
```

- `.str` is a string accessor in pandas
- `lower()` converts text to lowercase
- `upper()` converts text to uppercase
- Helps avoid mismatches due to case differences

Removing leading and trailing spaces

Unwanted spaces may exist in text data and should be removed.

```
df['label'] = df['label'].str.strip()
```

- `strip()` removes extra spaces from beginning and end
- Improves consistency in string comparison

Replacing values in strings

String values can be replaced to standardize data.

```
df['label'] = df['label'].str.replace('inbox', 'received')
```

- Replaces one string value with another
- Useful for standard naming
- Improves clarity of categorical data

Checking string patterns using conditions

String manipulation can be used to create conditions.

```
df['is_sent'] = df['label'].str.contains('sent')
```

- `contains()` checks whether a string contains a pattern
- Returns True or False
- Used to compute indicators

Extracting part of a string

Useful information can be extracted from strings.

```
df['domain'] = df['email'].str.split('@').str[1]
```

- `split()` divides string into parts
- Extracts domain name from email address
- Creates a new transformed column

String length calculation

Length of string values can also be computed.

```
df['length'] = df['subject'].str.len()
```

- Calculates number of characters
- Useful for analyzing text size or content